Суперкомпьютерное моделирование и технологии

Задание 1, вариант 18. Этап 1: OpenMP-программа

Хроян Георгий Овикович, группа 609

Октябрь 2026

1. Постановка задачи

В области

$$D = \{(x, y) : -1 < x, y < 1\} \setminus \{(x-1)^2 + (y-1)^2 \le 1\}$$

(квадрат без четверти круга с центром в $(1, 1)$) требуется приближённо решить задачу Дирихле

$$-\Delta u = 1, \quad (x, y) \in D, \qquad u = \sin(x + y), \quad (x, y) \in \gamma = \partial D.$$

Метод фиктивных областей. Задача заменяется задачей в квадрате $\Pi = (-1, 1)^2$:

$$-\Delta v + Q(x, y)\,(v - v_0) = F(x, y), \qquad v\big|_{\partial\Pi} = v_0, \qquad v_0 = \sin(x + y),$$

где $Q = 0$, $F = 1$ в D и $Q = 1/\varepsilon^2$, $F = 0$ в фиктивной области $\hat{D} = \Pi \setminus \overline{D}$; $\varepsilon = h^2$, $h = \max(h_1, h_2)$. В $\hat{D}$ большой коэффициент Q прижимает решение к v_0, и граничное условие на дуге выполняется приближённо без явного задания границы.

Разностная схема. Сетка $x_i = -1 + ih_1$, $y_j = -1 + jh_2$, $h_1 = 2/M$, $h_2 = 2/N$. Во внутренних узлах

$$-w_{\bar{x}x,ij} - w_{\bar{y}y,ij} + Q_{ij} w_{ij} = F_{ij} + Q_{ij} v_0(x_i, y_j),$$

на границе квадрата $w_{ij} = v_0(x_i, y_j)$. Коэффициенты усредняются по клетке $\Pi_{ij} = [x_i \pm h_1/2] \times [y_j \pm h_2/2]$: при доле площади клетки внутри D, равной $s_{ij} = \text{mes}(\Pi_{ij} \cap D)/(h_1 h_2)$, получаем $F_{ij} = s_{ij}$, $Q_{ij} = (1 - s_{ij})/\varepsilon^2$. Площадь пересечения клетки с кругом вычисляется точно (через первообразную $\sqrt{R^2 - x^2}$ и включение-исключение по углам клетки).

Метод решения. Метод сопряжённых градиентов с диагональным предобуславливателем $D_{ij} = 2/h_1^2 + 2/h_2^2 + Q_{ij}$, начальное приближение $w^{(0)} = 0$, скалярное произведение $(u, v) = \sum h_1 h_2 u_{ij} v_{ij}$, остановка при $\|w^{(k+1)} - w^{(k)}\|_E < \delta = 10^{-6}$, не более $(M-1)(N-1)$ итераций.

2. Последовательная программа

Сеточные функции хранятся одномерными массивами размера $(M+1)(N+1)$, узел (i, j) имеет индекс $i(N+1)+j$. Граничные узлы содержат v_0 и не меняются; невязка, направление и Ap на границе равны нулю. Одна итерация CG — три прохода по сетке: вычисление Ap вместе с (Ap, p) и (p, p); обновление w, r вместе с новым (z, r); новое направление $p = z + \beta p$.

Последовательная версия прошла четыре попытки на Polus (каталог `01_serial/runs`):

В попытке 4 массивы размещены в одном блоке памяти со сдвигом 13 КБ между ними, чтобы элементы с одним индексом не попадали в одни наборы кэша L1 (64 КБ) ядра POWER8,

Таблица 1. История последовательной версии на Polus.

№	Изменение	Итерации, 80×80	Время, с, 800×1200
1	без предобуславливателя, Q и F по узлу	305	—
2	+ диагональный предобуславливатель	168	—
3	+ точная площадь клетки (совпадение с эталоном)	167	29.98 (c2)
4	+ один блок памяти со сдвигом, готовая диагональ	167	17.74 (c4), 29.20 (c2)

а диагональ D хранится готовой. Время попыток 3 и 4 в таблице измерено на разных узлах (polus-c2 и polus-c4). Повторный замер попытки 4 на том же узле polus-c2 дал 29.20 с против 29.98 с у попытки 3: выигрыш от этих изменений около 3%, а основная разница в таблице объясняется тем, что узел polus-c2 примерно в 1.65 раза медленнее polus-c4 на этой задаче. Поэтому сравнивать время можно только в пределах одного узла, и ускорение в таблице 5 считается относительно последовательной программы, запущенной в том же задании.

3. OpenMP-программа

Программа разделена на три файла: `problem.h` — геометрия области, точная площадь $\mathrm{mes}(\Pi_{ij} \cap D)$ и коэффициенты (11); `solver.h` — сетка, заполнение коэффициентов и метод сопряжённых градиентов; `solver.cpp` — разбор аргументов, замер времени и вывод. Тесты (`test.cpp`) используют те же `problem.h` и `solver.h`.

Пять циклов по узлам сетки (подготовка коэффициентов, старт CG и три цикла итерации) размечены директивой `parallel for`: строки i делятся между нитями поровну. Скалярные произведения считаются через `reduction`, каждая нить копит свою частичную сумму:

```
#pragma omp parallel for reduction(+:pAp, pp)
for (int i = 1; i < M; i++)
    for (int j = 1; j < N; j++) {
        int k = i * s + j;
        Ap[k] = D[k] * p[k] - ih1 * (p[k - s] + p[k + s]) - ih2 * (p[k - 1] + p[k + 1]);
        pAp += Ap[k] * p[k];
        pp += p[k] * p[k];
    }
```

Итерации CG выполняются последовательно; α, β и проверка остановки вычисляются одной нитью между параллельными циклами. Нити привязаны к ядрам переменной окружения `GOMP_CPU_AFFINITY` (GCC 4.8.5 на Polus не поддерживает `OMP_PLACES`): каждая нить — на первый аппаратный поток своего ядра; задача занимает узел целиком (`#BSUB -x`, 16 ядер POWER8). При 32 нитях — по две нити на ядро.

История попыток (каталог `02_openmp/runs`):

1. `parallel for` без `reduction`: гонка данных на суммах, на 4 и 16 нитях метод не сходится (лимит 6241 итерация, $w(0,0)$ до $-9 \cdot 10^8$).
2. `reduction`: решение верное, но на 400×600 ускорение лишь $\times 4.9$ на 16 нитях.
3. привязка нитей к ядрам и эксклюзивный узел: $\times 12.5$ на 16 нитях (рис. 1).
4. код разделён на `problem.h`/`solver.h`/`solver.cpp`, добавлены тесты и запись истории итераций (норма изменения решения, невязка, ошибка относительно эталона).

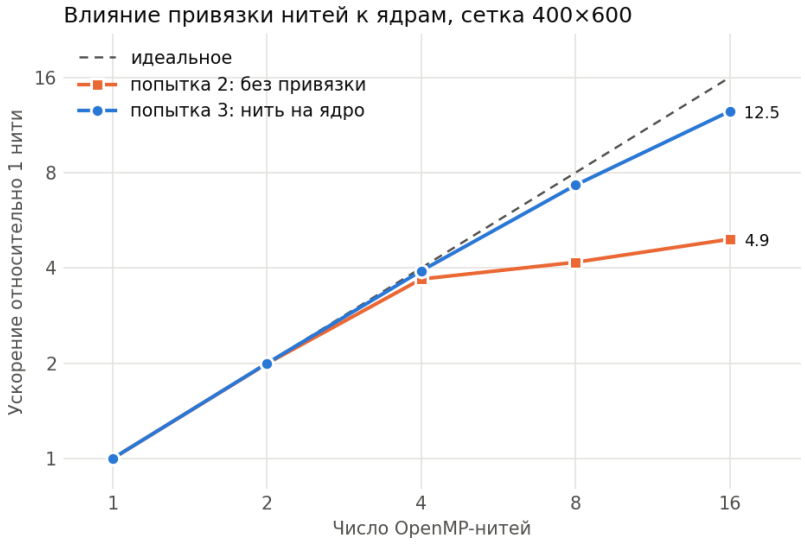


Рис. 1. Ускорение на сетке 400×600 без привязки нитей (попытка 2) и с привязкой (попытка 3).

4. Тестирование

Тесты собираются и запускаются командой `make test` (файл `test.cpp`):

Таблица 2. Тесты программы.

Что проверяется	Результат
Площадь: весь квадрат Π	$\mathrm{mes}(D) = 4 - \pi/4$ с точностью 10^{-15}
Площадь: угол $[0,1]^2$, левая половина, клетка внутри круга	$1 - \pi/4$, 2, 0
Площадь: сумма по клеткам сетки 80×80	$4 - \pi/4$, ошибка 10^{-13}; $0 \le S_{ij} \le h_1 h_2$
Эталон: сетки 20, 40, 80	44, 85, 167 итераций; $w(0,0)$ совпадает с независимой реализацией
Точное решение $u = \sin(x+y)$ ($Q = 0$, $F = 2\sin(x+y)$)	ошибка $1.34 \cdot 10^{-4}$, $3.36 \cdot 10^{-5}$, $8.43 \cdot 10^{-6}$ — убывает в 3.98 и 3.99 раза при удвоении сетки (второй порядок схемы)
Фиктивная область: узлы с $S_{ij} = 0$	$\max \lvert w - v_0 \rvert = 2.0 \cdot 10^{-4}$, $1.2 \cdot 10^{-5}$, $7.8 \cdot 10^{-7}$ — убывает при сгущении
Нити: 1, 2, 4, 8 нитей, сетка 80×80	одинаковое число итераций, $\max \lvert w_T - w_1 \rvert = 2 \cdot 10^{-15}$
История итераций	одна запись на итерацию, остановка при $\lVert w^{(k+1)} - w^{(k)} \rVert_E < \delta$

5. Сходимость по итерациям

Программа с ключом `--history` записывает для каждой итерации $\|w^{(k+1)} - w^{(k)}\|_E$ и невязку $\|r^{(k)}\|_E$ (рис. 2). Ошибка по итерациям (рис. 3) показана для двух задач: для задачи варианта 18 — относительно решения w^* той же сетки, полученного с $\delta = 10^{-12}$ (ключ `--ref`); для тестовой задачи с точным решением $u = \sin(x+y)$ (ключ `--exact`) — относительно u.

На тестовой задаче ошибка сначала падает вместе с итерационной, а затем выходит на уровень ошибки разностной схемы: $8.4 \cdot 10^{-6}$, $2.4 \cdot 10^{-7}$, $6.2 \cdot 10^{-8}$ на сетках 80×80, 400×600, 800×1200. Число итераций растёт с размером сетки примерно пропорционально $1/h$, как и ожидается для метода сопряжённых градиентов с диагональным предобуславливателем.

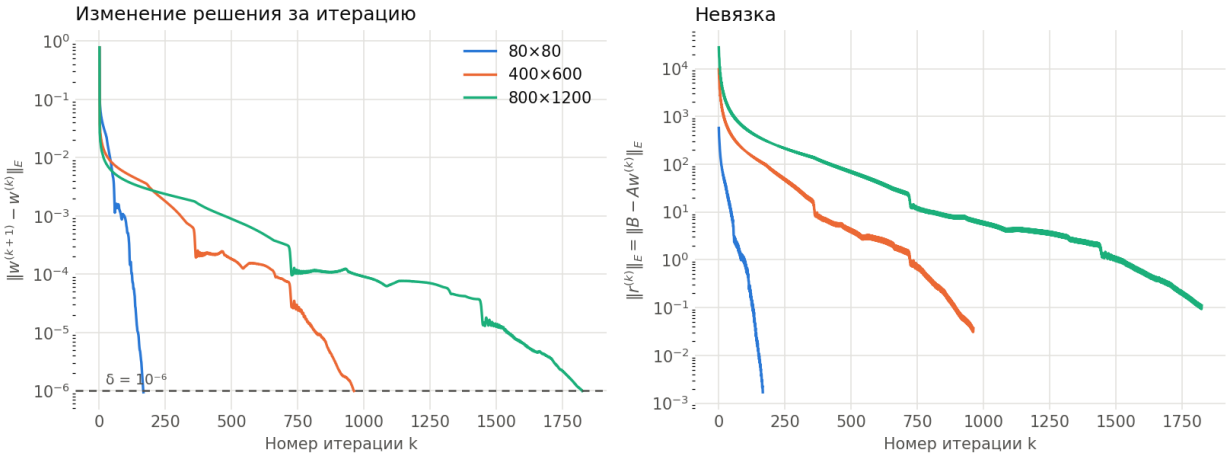


Рис. 2. Изменение решения за итерацию и невязка, задача варианта 18, $\delta = 10^{-6}$.

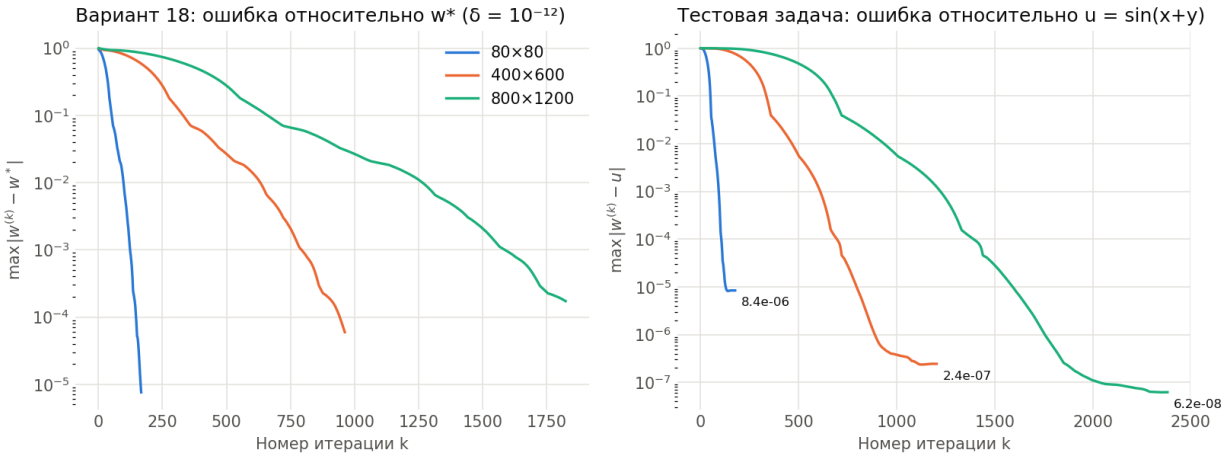


Рис. 3. Ошибка по итерациям в норме C. Слева — задача варианта 18 относительно w^* ($\delta = 10^{-12}$); справа — тестовая задача относительно точного решения ($\delta = 10^{-10}$).

Для задачи варианта 18 при остановке по $\delta = 10^{-6}$ итерационная ошибка в норме C равна $7.6{\cdot}10^{-6}$, $6.0{\cdot}10^{-5}$ и $1.7{\cdot}10^{-4}$ на трёх сетках. На самой мелкой сетке она сравнима с изменением решения при сгущении сетки, поэтому для точных расчётов на ней стоит брать δ меньше; для замеров времени (таблица 5) используется $\delta = 10^{-6}$.

6. Результаты

Все расчёты выполнены на IBM Polus в одном задании на эксклюзивном узле polus-c2, каждая конфигурация дважды, в таблицах — минимальное время. Ускорение $S = T_{\text{serial}}/T_{\text{OpenMP}}$ на той же сетке.

Проверка на сетке 80×80 (табл. 3).

Таблица 3. OpenMP против последовательной программы, сетка 80×80.

Нити	Итерации	Время, с	Ускорение	$w(0,0)$	$\lvert w - w_{\text{serial}}\rvert$
1	167	0.0181	1.00	0.241097385417	0
4	167	0.0064	2.85	0.241097385417	0
16	167	0.0153	1.19	0.241097385417	0

Решение OpenMP-программы совпадает с последовательным на любом числе нитей. На сетке 80×80 мало работы, и накладные расходы на запуск и синхронизацию нитей сравнимы с вычислениями, поэтому 16 нитей медленнее 4.

Время последовательной программы, относительно которого считается ускорение, приведено в табл. 4, результаты OpenMP-программы — в табл. 5 и на рис. 4.

Таблица 4. Последовательная программа (база для ускорения).

Сетка $M \times N$	Итерации	Время, с	$w(0,0)$
80×80	167	0.0182	0.241097385
400×600	962	3.8525	0.243036155
800×1200	1824	29.2048	0.243249324

Таблица 5. Результаты расчётов на ПВС IBM Polus (OpenMP-код).

Нити	Сетка $M \times N$	Итерации	Время, с	Ускорение
2	400×600	962	1.8741	2.06
4	400×600	962	0.9455	4.07
8	400×600	962	0.4911	7.85
16	400×600	962	0.3170	12.15
32	400×600	962	0.3447	11.18
2	800×1200	1824	14.1422	2.07
4	800×1200	1824	7.0917	4.12
8	800×1200	1824	3.5805	8.16
16	800×1200	1824	1.9253	15.17
32	800×1200	1824	1.7314	16.87

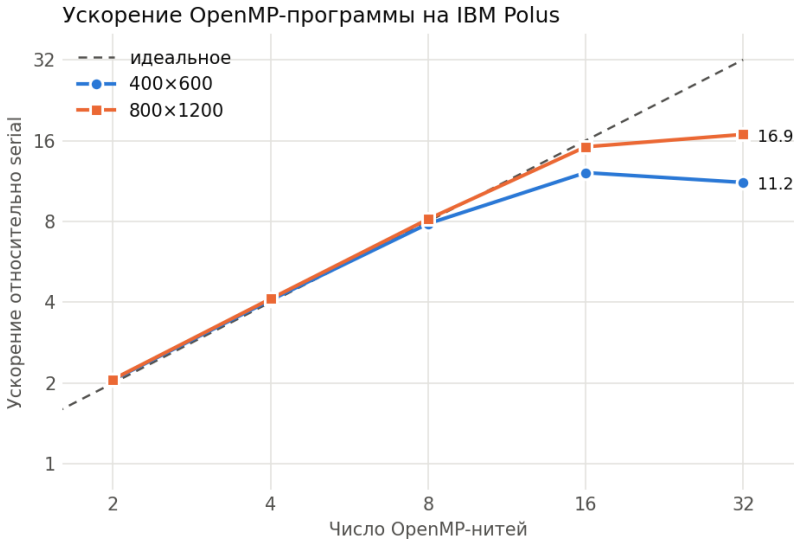


Рис. 4. Ускорение OpenMP-программы относительно последовательной.

Сравнение сеток (рис. 5–7). На грубых сетках видна ступенчатая аппроксимация дуги γ клетками сетки; при сгущении граница сглаживается, а значение $w(0,0)$ устанавливается около 0.2432. Немонотонность на грубых сетках объясняется тем, что ошибка метода фиктивных областей зависит от того, как дуга проходит через клетки, и не убывает строго как $O(h^2)$.

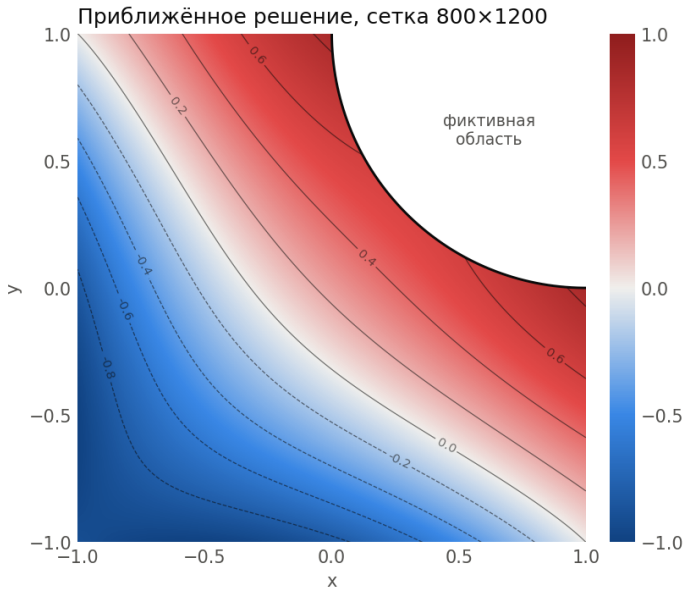


Рис. 5. Приближённое решение на сетке 800×1200; фиктивная область не показана, дуга — граница γ.

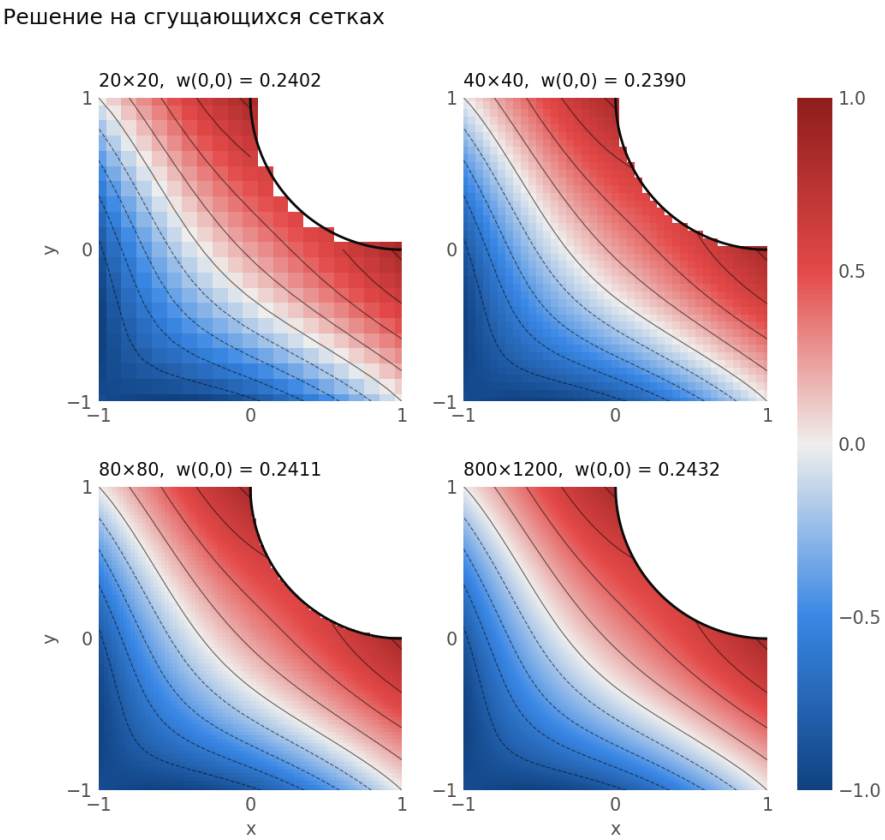


Рис. 6. Решение на сетках 20×20, 40×40, 80×80 и 800×1200 в одной шкале.

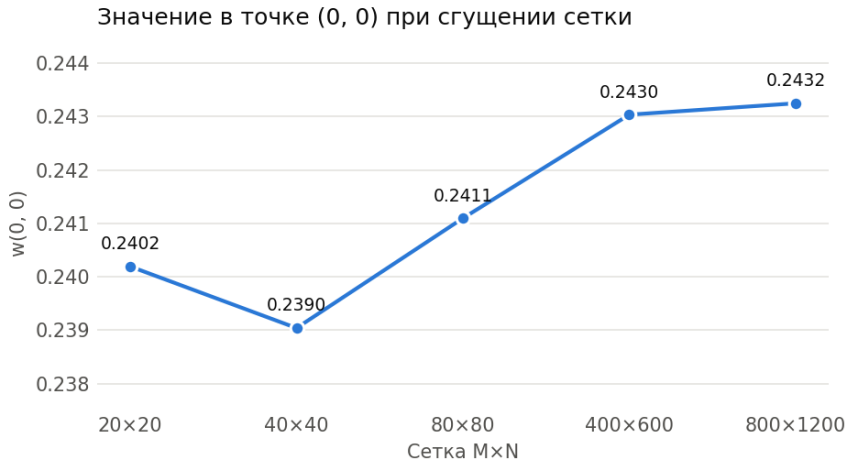


Рис. 7. Значение $w(0,0)$ на сгущающихся сетках (последовательная программа, Polus).

7. Выводы

- Последовательная и OpenMP-программы дают одинаковое решение; на сгущающихся сетках $w(0,0)$ устанавливается около 0.2432.
- До 16 нитей ускорение близко к линейному; на большей сетке оно выше, так как доля накладных расходов меньше.
- Две нити на ядро (32 нити на 16 ядрах) на сетке 800×1200 дают небольшой прирост ($\times 16.9$ против $\times 15.2$), а на 400×600 замедляют ($\times 11.2$ против $\times 12.2$): нити делят вычислительные блоки и кэш ядра, метод ограничен пропускной способностью памяти, а на меньшей сетке накладные расходы на синхронизацию 32 нитей перевешивают.
- Решающим для масштабирования оказалась привязка нитей к ядрам: без неё на 16 нитях ускорение было в 2.5 раза ниже.
- Узлы Polus заметно различаются по скорости: тот же последовательный код на polus-c2 работает в 1.65 раза дольше, чем на polus-c4. Ускорение от этого почти не зависит (на polus-c4 — $\times 15.0$ на 16 нитях для 800×1200), но абсолютное время сравнимо только в пределах узла.
- Тесты подтверждают точную площадь пересечения, второй порядок схемы на задаче с точным решением и совпадение результатов на любом числе нитей.